Algorithm: Minimum Subset Sum Difference

1. **Initialize totals**
    - Compute the total sum of all elements in the array (sum2).
    - Start with sum1 = 0 and idx = 0.
2. **Define recursive function** solve(sum1, sum2, idx)
    - If idx reaches the end of the array, return the absolute difference |sum1 - sum2|.
    - If the state (idx, sum1) is already computed in dp, return the stored value.
3. **Recursive choices**
    - **Pick**: Add arr[idx] to subset1 (sum1 + arr[idx]) and subtract from subset2 (sum2 - arr[idx]).
    - **Not Pick**: Leave arr[idx] in subset2, keeping sum1 unchanged.
    - Compute both results recursively.
4. **Store and return**
    - Take the minimum of the two results.
    - Save it in dp[idx][sum1] to avoid recomputation.
    - Return this value.
5. **Final answer**
    - Call solve(0, sum2, 0) with the initial sums.
    - The result is the minimum possible difference between two subset sums.

Complexity

- **Time Complexity**: $O(n \cdot S)$, where $n$is the number of elements and $S$is the total sum of the array.
- **Space Complexity**: $O(n \cdot S)$for the DP table.